KONFIGURASI INDEX PADA POSTGRESQL

Disusun Untuk Memenuhi Tugas Pertemuan 1

Mata Kuliah : Prak. Rekayasa dan Arsitektur Basis Data

Dosen Pengampu:

Cecep Kurnia Sastradipraja, S.Kom., M.Kom.



Disusun Oleh:

Pirda Nur Awaliah 2507066

Sistem Informasi B

JURUSAN ILMU KOMPUTER

PROGRAM STUDI SISTEM INFORMASI

INSTITUT TEKNOLOGI GARUT

2026

Konfigurasi Index pada PostgreSQL

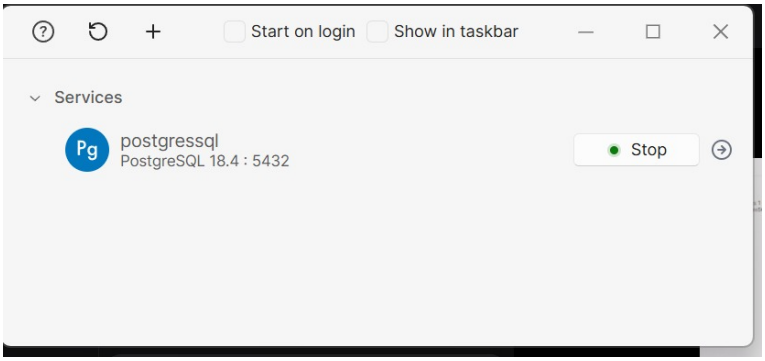
Tujuan Praktikum

Praktikum ini bertujuan untuk memahami fungsi index pada PostgreSQL, mengetahui cara membuat dan memeriksa index, serta memahami penggunaan single-column dan composite index untuk meningkatkan efisiensi pencarian data.

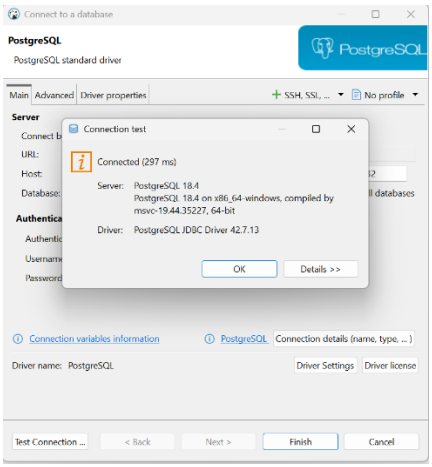
Dibuatnya ini untuk mengerjakan tugas Praktikum Rekayasa & Arsitektur Basis Data.

Langkah-langkah pengerjaan:

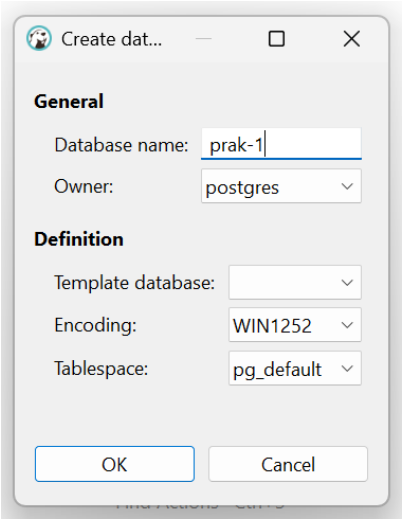
1. Buka DBngin, konek ke postgresql, lalu buat Database di DBngin, dan klik start, seperti Gambar di bawah



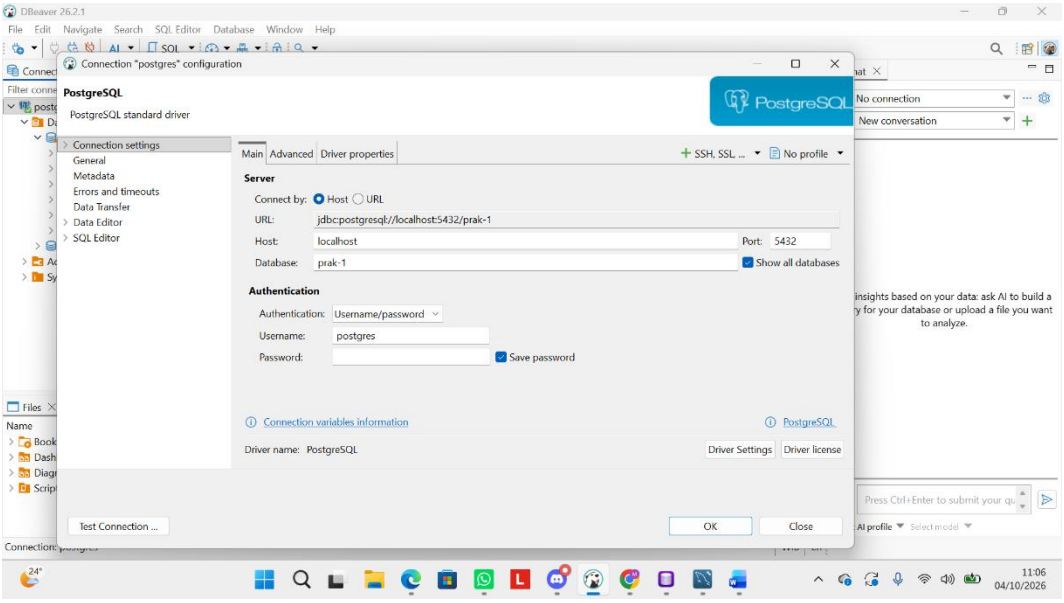
2. Buka DBeaver, lalu konek ke postgresql, lalu test Connection dulu, kalau aman lanjut finish.



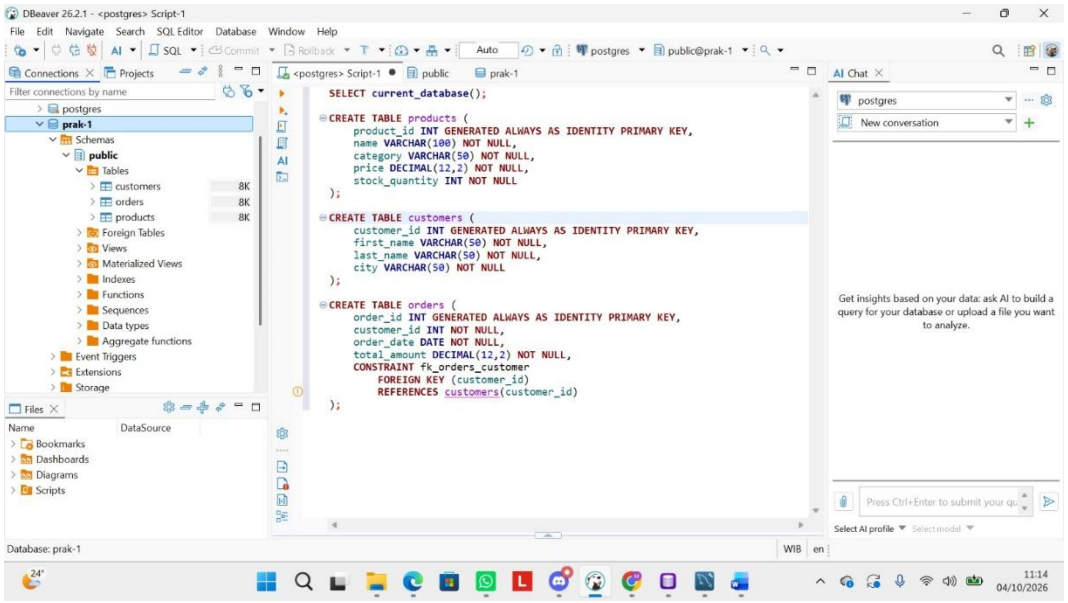
3. Lalu create database baru di dbeaver, agar sesuai nama Praktikum, contoh prak-1, lalu OK.



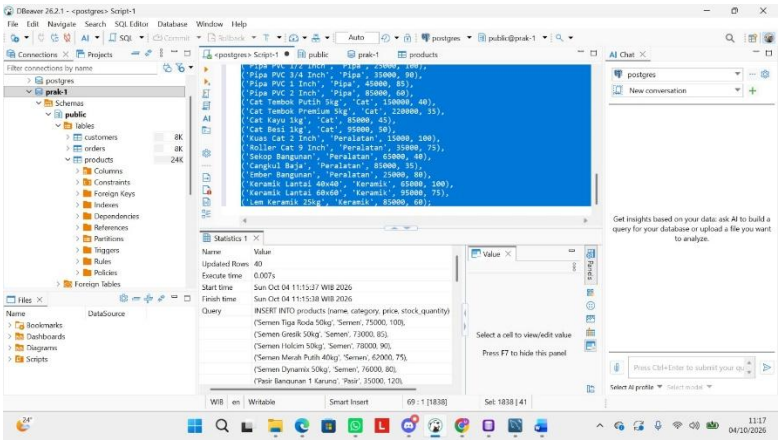
4. Lalu edit connection database default sebelumnya, Ganti nama database jadi prak-1, karena database prak-1 sudah eksis/ada, maka bisa konek loh ya, lalu OK.

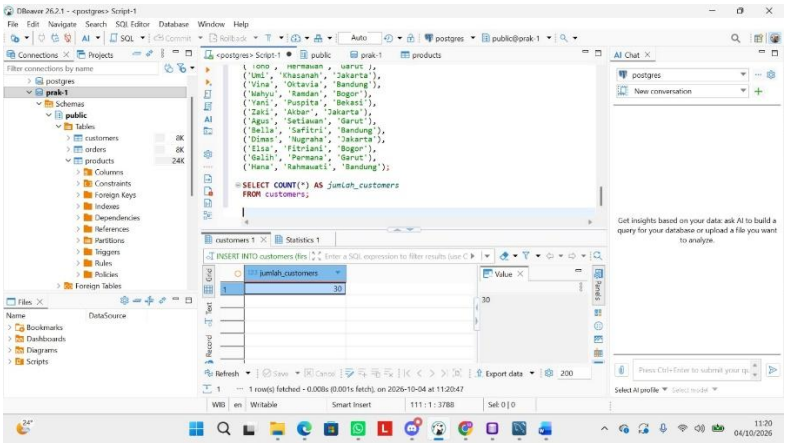


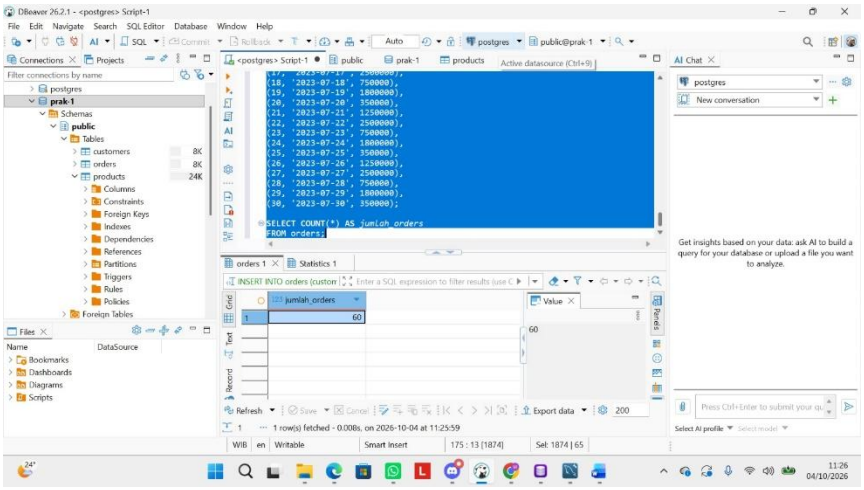
5. Lalu open SQL Editor, dan masukkan querry seperti digambar untuk tahap pembuatan Table.



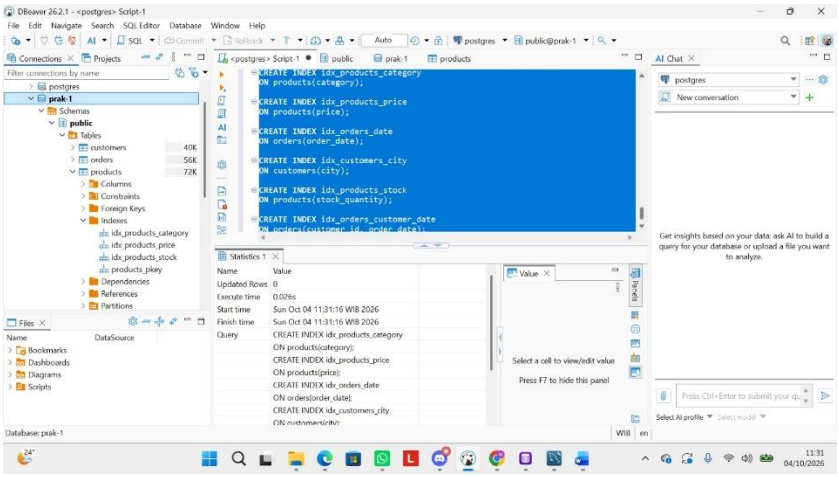
6. Lalu lanjut ke penginputan data set, 40 products, 30 costumers, dan 60 orders, isinya kami isi dengan data set dari toko matrial sebagai contoh.

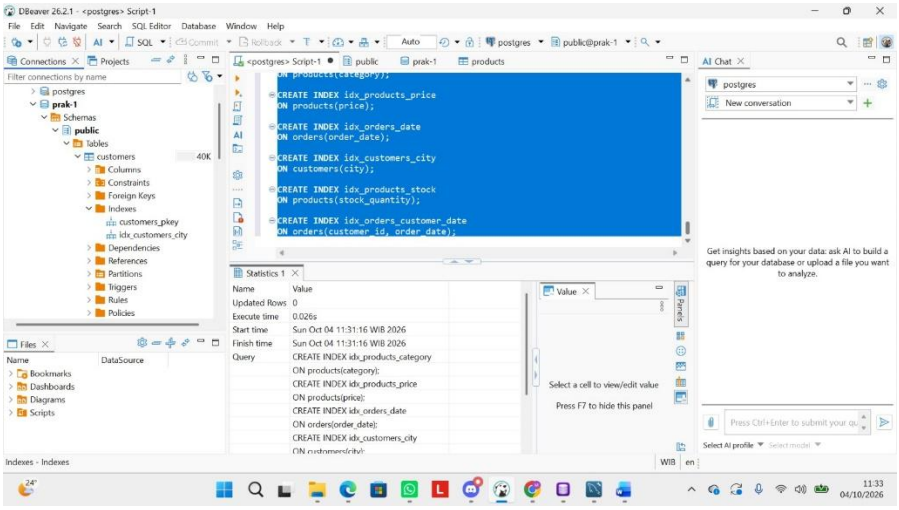


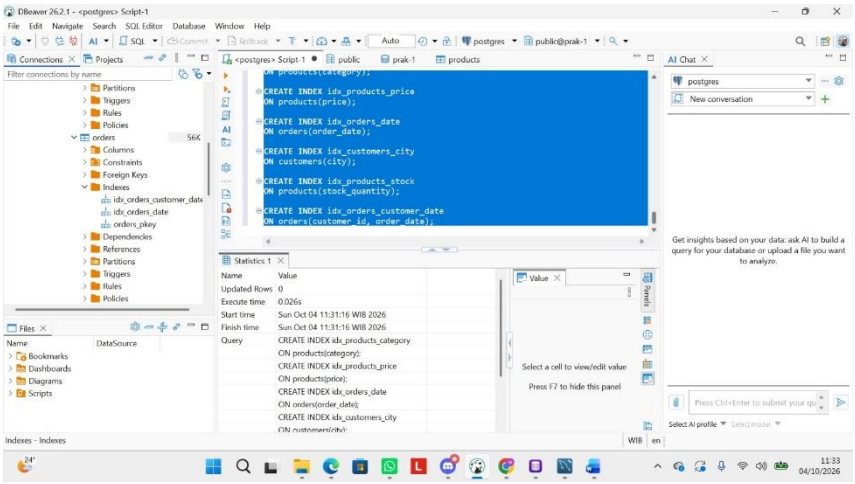

DBeaver 26.2.1 - <postgres> Script-1
File Edit Navigate Search SQL Editor Database Window Help
Connections Projects
postgres
prak-1
Schemas
public
Tables
customers 8K
orders 8K
products 24K
Columns
Constraints
Foreign Keys
Indexes
Dependencies
References
Partitions
Triggers
Rules
Policies
Foreign Tables
Files
Name DataSource
Bookmarks
Dashboards
Diagrams
Scripts
<postgres> Script-1 public prak-1 products
('Umi', 'Khasanah', 'Jakarta'),
('Vina', 'Oktavia', 'Bandung'),
('Wahyu', 'Ramdan', 'Bogor'),
('Yani', 'Puspita', 'Bekasi'),
('Zaki', 'Akbar', 'Jakarta'),
('Agus', 'Setiawan', 'Garut'),
('Bella', 'Safitri', 'Bandung'),
('Dimas', 'Nugraha', 'Jakarta'),
('Elsa', 'Fitriani', 'Bogor'),
('Galih', 'Permana', 'Garut'),
('Hana', 'Rahmawati', 'Bandung');
SELECT COUNT(*) AS jumlah_customers
FROM customers;
customers 1 Statistics 1
jumlah_customers
1 30
Value
30
Refresh Save Cancel Export data 200
1 row(s) fetched - 0.008s (0.001s fetch), on 2026-10-04 at 11:20:47
WIB en Writable Smart Insert 111 : 1 : 3788 Sel: 0 | 0
AI Chat
postgres
New conversation
Get insights based on your data: ask AI to build a query for your database or upload a file you want to analyze.
Press Ctrl+Enter to submit your m
Select AI profile
11:20
04/10/2026



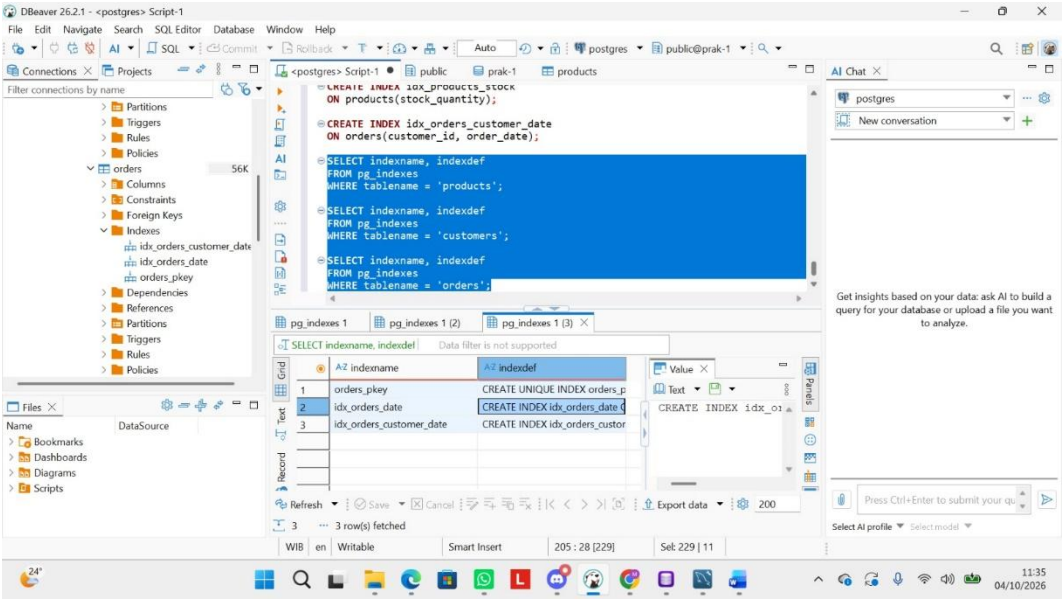
7. Sekarang kita lanjut ke tahap pembuatan index Products, Costumers dan Indexs, untuk Querry nya nanti ada di bawah.







8. Oke, setelah pembuatan indexs, lanjut ke pemeriksaan Indexs.



9. Oke sekarang saya akan menjawab pertanyaan Diskusi dari Modul

1. Apa fungsi index pada basis data?

Index berfungsi untuk membantu DBMS menemukan data tertentu secara lebih efisien tanpa harus membaca seluruh baris dalam tabel. Index dapat mempercepat proses pencarian data menggunakan kondisi WHERE, operasi JOIN, serta operasi tertentu seperti ORDER BY dan GROUP BY, bergantung pada struktur index yang digunakan.

2. Apakah setiap kolom harus diberikan index? Jelaskan.

Tidak, karena tidak semua kolom sering digunakan untuk pencarian atau pengolahan data. Index sebaiknya dibuat pada kolom yang sering digunakan dalam query, seperti kolom yang digunakan pada kondisi WHERE atau JOIN. Pembuatan index pada setiap kolom justru dapat menambah kebutuhan penyimpanan dan biaya pemeliharaan database.

3. Mengapa index dapat memperbesar biaya INSERT, UPDATE, dan DELETE?

Karena setiap kali data ditambahkan, diubah, atau dihapus, PostgreSQL juga perlu memperbarui struktur index yang berkaitan dengan data tersebut. Proses pemeliharaan ini membutuhkan sumber daya dan waktu tambahan, sehingga semakin banyak index yang dimiliki suatu tabel, semakin besar pula biaya yang diperlukan untuk operasi perubahan data.

4. Apa perbedaan single-column index dan composite index?

Single-column index adalah index yang dibuat pada satu kolom dalam tabel, sedangkan composite index adalah index yang dibuat menggunakan dua kolom atau lebih.

Contoh pada database toko material:

```
-- Single-column index
CREATE INDEX idx_products_category
ON products(category);

-- Composite index
CREATE INDEX idx_orders_customer_date
ON orders(customer_id, order_date);
```

Index pertama hanya menggunakan kolom category, sedangkan index kedua menggunakan kolom customer_id dan order_date secara bersamaan.

5. Mengapa urutan kolom pada composite index penting?

Urutan kolom pada composite index menentukan pola pencarian yang dapat didukung oleh index tersebut. Misalnya, index (customer_id, order_date) lebih sesuai untuk query yang mencari berdasarkan customer_id, atau berdasarkan customer_id sekaligus rentang order_date. Susunan (order_date, customer_id) tidak identik karena urutan kolom awalnya berbeda, sehingga pemanfaatannya untuk query juga dapat berbeda.

6. Bagaimana index dapat membantu operasi JOIN?

Index dapat membantu operasi JOIN dengan mempercepat pencarian baris yang memiliki nilai kolom penghubung yang sesuai pada tabel lain. Contohnya, pada tabel orders terdapat kolom customer_id sebagai foreign key yang menghubungkan data pesanan dengan tabel customers. Index pada kolom yang digunakan untuk menghubungkan kedua tabel dapat membantu PostgreSQL menemukan data yang cocok dengan lebih efisien. Namun, PostgreSQL tidak otomatis membuat index pada kolom foreign key.

7. Apa risiko membuat terlalu banyak index pada suatu tabel?

Terlalu banyak index dapat meningkatkan penggunaan ruang penyimpanan dan biaya pemeliharaan database. Setiap operasi INSERT, UPDATE, dan DELETE juga dapat membutuhkan proses tambahan untuk memperbarui index yang terkait. Oleh karena itu, index perlu dibuat sesuai kebutuhan query dan tidak berlebihan agar pengelolaan database tetap efisien.

KESIMPULAN

Berdasarkan praktikum konfigurasi indexing pada PostgreSQL yang telah dilakukan, database toko material berhasil dirancang dengan tiga tabel, yaitu products, customers, dan orders, yang saling berhubungan untuk menyimpan data produk, pelanggan, dan transaksi. Dataset yang digunakan terdiri dari 40 data produk material bangunan, 30 data pelanggan, dan 60 data pesanan sesuai dengan ketentuan praktikum.

Pada praktikum ini, telah dirancang *single-column index* pada kolom category dan price di tabel products, serta *composite index* pada kolom customer_id dan order_date di tabel orders. Index tersebut disesuaikan dengan pola query yang digunakan, seperti pencarian produk berdasarkan kategori atau rentang harga, serta pencarian pesanan

berdasarkan pelanggan dan tanggal transaksi. Pemeriksaan melalui pg_indexes digunakan untuk melihat index yang telah dibuat pada tabel.

Dari hasil praktikum ini, dapat disimpulkan bahwa index merupakan salah satu komponen penting dalam pengelolaan basis data karena dapat membantu proses pencarian dan pengambilan data menjadi lebih efisien. Namun, pembuatan index perlu disesuaikan dengan kebutuhan query karena setiap index juga membutuhkan ruang penyimpanan dan pemeliharaan ketika data ditambahkan, diubah, atau dihapus. Dengan demikian, pemilihan jenis dan kolom index yang tepat dapat membantu pengelolaan database toko material secara lebih terstruktur dan efisien.

REFLEKSI

1. Index mana saja yang berhasil dibuat, dan pada kolom apa?

Index yang dibuat pada database toko material terdiri dari dua *single-column index* dan satu *composite index*. *Single-column index* yang dibuat adalah idx_products_category pada kolom category dan idx_products_price pada kolom price di tabel products. Sementara itu, *composite index* yang dibuat adalah idx_orders_customer_date pada kolom customer_id dan order_date di tabel orders. Index tersebut dibuat untuk mendukung pencarian data berdasarkan kategori produk, harga, serta pelanggan dan tanggal pesanan.

2. Apa konsekuensi jika sebuah tabel memiliki terlalu banyak index?

Jika sebuah tabel memiliki terlalu banyak index, kebutuhan ruang penyimpanan database akan meningkat karena setiap index membutuhkan ruang tersendiri. Selain itu, operasi INSERT, UPDATE, dan DELETE dapat menjadi lebih lambat karena PostgreSQL harus memperbarui index yang berkaitan dengan perubahan data. Oleh karena itu, jumlah index perlu disesuaikan dengan kebutuhan agar tidak menimbulkan biaya pemeliharaan yang berlebihan.

3. Bagaimana menentukan apakah sebuah kolom layak diberikan index?

Kolom layak diberikan index apabila sering digunakan dalam query, terutama pada kondisi WHERE, operasi JOIN, atau pengurutan data menggunakan ORDER BY. Pemilihan index juga perlu mempertimbangkan pola pencarian dan kebutuhan aplikasi. Pada database toko material, kolom category dan price diberikan index karena digunakan

untuk pencarian produk berdasarkan kategori dan rentang harga. Dengan demikian, index sebaiknya dibuat berdasarkan kebutuhan query dan bukan pada semua kolom tabel.